

PitchSwitch — Architecture

snig-17

June 27, 2026

Architecture

An explanation doc (Diataxis): the *why* behind PitchSwitch's structure. For what it is and how to run it, see [README](#); for the visual system, [DESIGN.md](#).

The one thing to understand: there are two danger engines

PitchSwitch computes “how dangerous is this moment” in **two different places, for two different jobs**. Conflating them is the easiest way to misread the codebase, so this is the first thing to internalize.

ACCURACY ENGINE (backs the README numbers)
(what the hosted app runs)

```

StatsBomb events (gitignored / API)
tracking CSVs (gitignored)
    |
    core/replay.py    parse → MatchEvent
    |
load_frames() → Frame(t, ball, players)
    core/heat.py      rolling danger + rising-
ball depth + clustering, 0..1
    |                  danger derivative
    core/director.py  Granite tie-break on
build_unified_broadcast(matchups, favourite)
    |                  near-equal danger [NOT WIRED]
frame danger series + personalize bias
    |                  INTO THE LIVE DEMO]
deterministic differential switch schedule
    v
Granite narration per match (the "call")
    scripts/calibrate.py
rule events (Granite + Docling)
    → 47% / 71s / 42% @120s (README table)

contained <canvas>: avatars, danger glow,

LIVE DANGER panel, WHY line, Coach band

serves a pre-built cache (no Ollama/data)

Cloud (auto-deploy on push to main)

```

LIVE DEMO ENGINE

```

Metrica 25fps
    |
    core/metrica.py
    |
frame_danger() ←
    |
    • per-
    •
    • ONE
    • Coach

core/livefeed.py
    | self-
    | DIRECTOR —
    v
app.py Streamlit
    v
Streamlit Community

```

Accuracy engine — `heat.py` scores danger from StatsBomb event data (carries into the final third, box pressure, shot xG, set pieces) over a rolling 90s window, plus a time-derivative for “danger rising fast.” `scripts/calibrate.py` replays the demo matches through it to produce the published accuracy numbers. `director.py` is its companion: a two-tier switcher that calls IBM Granite to break near-equal danger ties.

Live demo engine — `metrica.frame_danger` scores danger from real 25fps player tracking (ball depth toward goal + how many players cluster at that end). The Director in the live feed cuts to whichever match's danger is **pulling ahead** (a deterministic differential), so a switch never waits on an LLM. Granite's job here is the *call*, not the *cut*: one grounded narration sentence per match. Coach adds rule explanations.

Why two engines instead of one

This wasn't the plan; it's the honest result of two constraints colliding:

1. **The accuracy story needs event data.** StatsBomb open data has xG, set pieces, and rich event types — enough to build a calibratable anticipation model and claim a real “~71s lead time” number. But StatsBomb has no continuous positions, so you can't render a watchable broadcast from it.
2. **The watchable broadcast needs tracking data.** Metrica's open data has all 22 players
 - ball at 25fps — real continuous movement you can animate. But it's anonymized and has no xG, so it can't reproduce the calibrated model.

So the project uses each dataset for what it's good at: StatsBomb for the *credibility* (measured accuracy), Metrica for the *experience* (the live feed). `director.py`'s Granite tie-break belongs to the StatsBomb engine and is **not wired into the hosted demo** — the live feed's switching is the tracking-derived differential.

Trade-offs (named explicitly)

- **The demo and the accuracy number run different models.** The headline “71s lead” is measured on `heat.py` /StatsBomb, not on the `metrica` danger the viewer watches. The README says this plainly rather than implying one number describes both.
- **Granite doesn't decide the live cut.** Decoupling the switch (deterministic, instant, visible) from the narration (Granite, async) keeps the broadcast real-time and never blocked on LLM latency. Cost: the marketed “Granite reasons over ambiguous switches” (the Director) is offline-only.

- **Self-contained cache over live compute.** `app.py` serves a pre-built broadcast cache (`data/cache/*.json`) so the hosted app needs no Ollama, no datasets, no API keys. Cost: a code change to the feed requires rebuilding the cache (`scripts/prebuild_cache.py`) for the hosted artifact to reflect it.

Where the three IBM technologies sit

- **Granite** (`providers/llm.py` , swappable Ollama/watsonx/Replicate) — writes the switch narration and Coach explanations in the live engine; the tie-break reasoner in the offline Director.
- **Docling** (`core/grounding.py` , `core/coach.py`) — parses team primers (`data/primers/`) and the Laws of the Game (`data/rules/`) into grounding for Granite.
- **Watson Text to Speech** (`providers/tts.py`) — speaks the narration and Coach lines; optional and graceful (silent if no creds).

Data flow at a glance

`favourite team(s)` → personalize bias → per-match `frame_danger` series → differential switch schedule → Granite narration + Coach events (Docling-grounded, TTS-voiced) → baked into a self-contained canvas → cached → served by Streamlit.

Related

- [README](#) — problem, solution, accuracy table, quick start
- [DESIGN.md](#) — the broadcast visual system
- [TESTING.md](#) — the test suite over the live path